

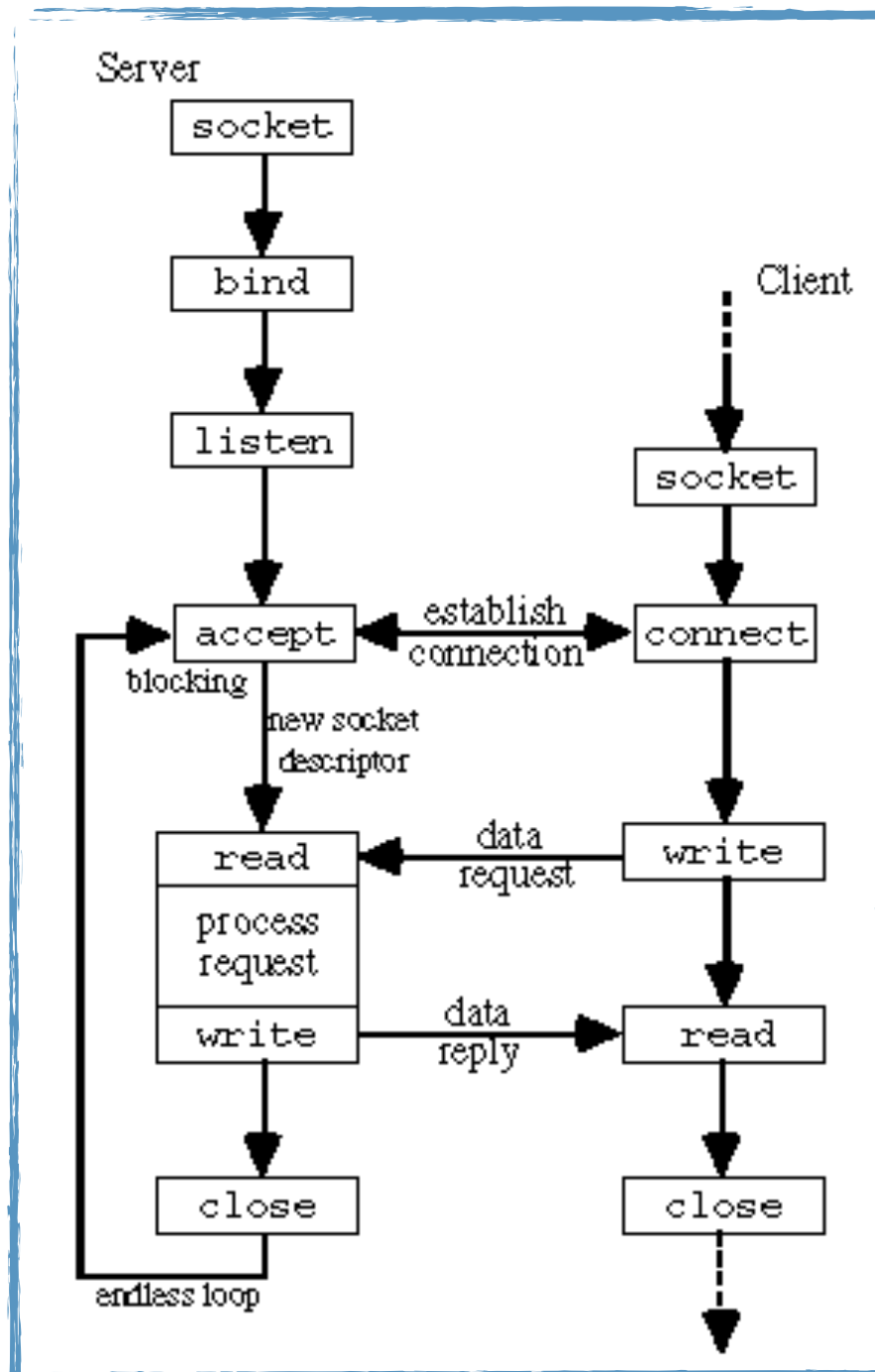
Laboratorul 6

Modelul Client/Server iterativ TCP.

Utilizarea primitivelor socket, bind, listen, connect, accept, close.

Exemplu de implementare.

1. Modelul Client/Server iterativ TCP



2. Utilizarea primitivei `socket()`

```
#include <sys/socket.h>
#include <sys/types.h>
```

```
int socket ( int domain, int type, int protocol )
```

Parametri:

- **domain** - specifica domeniul de comunicare unde va fi creat socket-ul si selecteaza familia de protocoale care va fi folosita pentru comunicare;
- **type** - tipul de socket care va fi creat, care determina si modalitatea in care se va comunica;
- **protocol** - cere sa fie folosit un protocol particular cu socket-ul creat (protocolul este de obicei obtinut din combinatia domain + type);

Intoarce 0 in caz de succes, -1 in caz de eroare si flag-ul `errno` este setat

Valori posibile argumente **socket**:

- domain
 - **AF_UNIX** - comunicare locala (in cadrul aceluiasi nod din retea);
 - **AF_INET** - comunicare in domeniul Internetului avand la nivelul Retea protocolul IP;
 - **AF_INET6** - comunicare in domeniul Internetului avand la nivelul retea protocolul IPv6;
- type

- **SOCK_STREAM** - comunicare secventiala, sigura, bidirectionala, prin stream-uri de bytes si poate oferi mecanisme de transmisie pentru datele out-of-band; este suportat in domeniile AF_INET, AF_INET6 si AF_UNIX;
- **SOCK_DGRAM** - comunicare prin datagrame, fara conexiune, transmisia mesajelor nu este sigura; mesajele au o lungime maxima fixa; este suportat in domeniile AF_INET, AF_INET6 si AF_UNIX;
- **SOCK_RAW** - ofera interfata de comunicare prin protocoale interne (cum ar fi IP sau ICMP); este suportat in domeniile AF_INET, AF_INET6; necesita privilegii de superuser pentru a putea fi folosit;
- protocol
 - 0 - este ales un protocol potrivit pentru tipul de socket-uri create;
 - 6 - **TCP (Transmission Control Protocol)**;
 - IPPROTO_TCP - **TCP (Transmission Control Protocol)**;
 - 17 - **UDP (User Datagram Protocol)**;
 - 132 - **SCTP (Stream Control Transmission Protocol)**;
- Exemple de **configuratii socket()**

Domain	Type	Protocol	Result
AF_INET	SOCK_STREAM	0	Selects TCP as communication protocol for the socket
AF_INET	SOCK_STREAM	6	Selects TCP as communication protocol for the socket
AF_INET	SOCK_DGRAM	0	Selects UDP as communication protocol for the socket pair

3. Utilizarea primitivelor *bind()*, *listen()*, *connect()*, *accept()*, *close()*

- **bind()** - asigneaza adresa specificata de *addr* la socket-ul referit prin descriptorul *sockfd*;

```
#include <sys/socket.h>
#include <sys/types.h>
```

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

// IPv4 AF_INET sockets structure:

```
struct sockaddr_in {
    short sin_family; // e.g. AF_INET, AF_INET6
    unsigned short sin_port; // e.g. htons(3490)
    struct in_addr sin_addr; // see struct in_addr, below
    char sin_zero[8]; // zero this if you want to
};
```

```
struct in_addr {
    unsigned long s_addr; // load with inet_pton()
};
```

// IPv4 bind example:

```
struct sockaddr_in ip4addr;
int s;

ip4addr.sin_family = AF_INET;
ip4addr.sin_port = htons(3490);
inet_pton(AF_INET, "10.0.0.1", &ip4addr.sin_addr);

s = socket(PF_INET, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&ip4addr, sizeof ip4addr);
```

- * Campul `s_addr` poate fi setat folosind constanta **INADDR_ANY** daca dorim sa facem bind pe adresa IP locala (**localhost** - **127.0.0.1**)

```
#include <arpa/inet.h>
```

```
int inet_pton(int af, const char *src, void *dst);
```

Parametri:

- `af` - valoarea unei familii de adrese (`AF_INET` sau `AF_INET6`);
- `src` - pointer la un string care contine adresa IP in forma afisabila;
- `dst` - referentiaza adresa unde ar trebui stocat rezultatul (cel mai frecvent o structura `in_addr` sau `in_addr6`;

- **listen()**

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);
```

Parametri:

- `sockfd` - descriptorul socketului pe care vrem sa facem ascultarea;
- `backlog` - numarul de conexiuni pending pe care le putem avea pana cand kernelul incepe sa le respinga pe cele noi;

- * Cand vin cereri noi de conexiune, ar trebui sa le dam `accept()` rapid astfel incat `backlog` sa nu treaca peste limita;

- **accept()**

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Parametri:

- sockfd - descriptorul socketului care asculta pentru noi conexiuni;
- addr - structura va fi populata cu informatii despre clientul care s-a conectat;
- Addrlen - contine dimensiunea structurii addr completate cu informatii despre client;

- **close()**

```
#include <unistd.h>
```

```
int close(int sockfd);
```

Parametri:

- sockfd - descriptorul socketului pe care vrem sa il inchidem;

- **connect()**

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int connect(int sockfd, const struct sockaddr *serv_addr, socklen_t addrlen);
```

4. Implementarea unui model client/server TCP/IP iterativ

Pentru a vizualiza un **exemplu de implementare a unui server TCP/IP iterativ** accesati urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/servTcpIt.c>

Pentru a descarca fisierul utilizati comanda `wget` intr-o sesiune activa de terminal, in urmatorul mod:

`wget https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/servTcpIt.c`

Pentru a vizualiza un **exemplu de implementare a unui client TCP/IP iterativ** accesati urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/cliTcpIt.c>

Pentru a descarca fisierul utilizati comanda `wget` intr-o sesiune activa de terminal, in urmatorul mod:

`wget https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/cliTcpIt.c`

Pentru a **compila** cele doua surse C, veti folosi un **Makefile** accesibil la urmatorul link:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/Makefile>

Pentru a descarca fisierul utilizati comanda `wget` intr-o sesiune activa de terminal, in urmatorul mod:

`wget https://profs.info.uaic.ro/~computernetworks/files/NetEx/S5/Makefile`